

Program 1

Aim:-

To implement a one-pass and two-pass assembler to translate assembly language into machine code.

Theory:-

Assemblers are tools that convert assembly language programs into machine code.

- **One-Pass Assembler:** Resolves labels and generates machine code in a single pass.
- **Two-Pass Assembler:** Processes the source code in two passes:
 - **Pass 1:** Builds the symbol table and calculates label addresses.
 - **Pass 2:** Uses the symbol table to generate the final machine code.

Code:-

Copy code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
typedef struct {
    char label[10];
    int address;
} Symbol;
```

```
Symbol symbolTable[10];
int symbolCount = 0;
```

```
typedef struct {
    char opcode[10];
    char operand[10];
    int address;
} Intermediate;
```

```
Intermediate intermediateCode[10];
```

```

int intermediateCount = 0;

// Add a label to the symbol table
void addSymbol(char *label, int address) {
    strcpy(symbolTable[symbolCount].label, label);
    symbolTable[symbolCount].address = address;
    symbolCount++;
}

// Get address of a label from the symbol table
int getAddress(char *label) {
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].label, label) == 0) {
            return symbolTable[i].address;
        }
    }
    return -1; // Symbol not found
}

// Display the symbol table
void displaySymbolTable() {
    printf("\nSymbol Table:\n");
    printf("Label\tAddress\n");
    for (int i = 0; i < symbolCount; i++) {
        printf("%s\t%d\n", symbolTable[i].label, symbolTable[i].address);
    }
}

// One-Pass Assembler
void onePassAssembler() {
    FILE *input = fopen("assembly.txt", "r");
    if (!input) {
        printf("Error: Cannot open assembly file.\n");
        return;
    }
}

```

```

    }

    char line[100], label[10], opcode[10], operand[10];
    int locationCounter = 0;

    printf("\nOne-Pass Assembler Output:\n");
    while (fgets(line, sizeof(line), input)) {
        if (line[0] != '\n') {
            sscanf(line, "%s %s %s", label, opcode, operand);

            if (strcmp(label, "-") != 0) {
                addSymbol(label, locationCounter);
            }

            printf("%d %s ", locationCounter, opcode);
            int address = getAddress(operand);
            if (address != -1) {
                printf("%d\n", address);
            } else {
                printf("%s\n", operand);
            }

            locationCounter += 1; // Assuming each instruction takes 1 unit
        }
    }

    fclose(input);
}

// Two-Pass Assembler: Pass 1
void passOne() {
    FILE *input = fopen("assembly.txt", "r");
    if (!input) {
        printf("Error: Cannot open assembly file.\n");
    }
}

```

```

        return;
    }

    char line[100], label[10], opcode[10], operand[10];
    int locationCounter = 0;

    while (fgets(line, sizeof(line), input)) {
        if (line[0] != '\n') {
            sscanf(line, "%s %s %s", label, opcode, operand);
            if (strcmp(label, "-") != 0) {
                addSymbol(label, locationCounter);
            }
            strcpy(intermediateCode[intermediateCount].opcode, opcode);
            strcpy(intermediateCode[intermediateCount].operand, operand);
            intermediateCode[intermediateCount].address = locationCounter;
            intermediateCount++;

            locationCounter += 1; // Assuming each instruction takes 1 unit
        }
    }

    fclose(input);
}

// Two-Pass Assembler: Pass 2
void passTwo() {
    printf("\nTwo-Pass Assembler Output:\n");
    for (int i = 0; i < intermediateCount; i++) {
        printf("%d %s ", intermediateCode[i].address, intermediateCode[i].opcode);

        int address = getAddress(intermediateCode[i].operand);
        if (address != -1) {
            printf("%d\n", address);
        } else {

```

```
        printf("%s\n", intermediateCode[i].operand);
    }
}
}
```

```
int main() {
    printf("Choose an Assembler:\n1. One-Pass Assembler\n2. Two-Pass\nAssembler\n");
    printf("Enter your choice: ");
    int choice;
    scanf("%d", &choice);

    if (choice == 1) {
        onePassAssembler();
    } else if (choice == 2) {
        passOne();
        displaySymbolTable();
        passTwo();
    } else {
        printf("Invalid choice.\n");
    }
    return 0;
}
```

Input File (assembly.txt):-

```
LABEL OPCODE OPERAND
- LOAD X
X DC 5
- ADD Y
Y DC 10
- STORE Z
Z DC 0
```

Sample Output:-

For One-Pass Assembler:

One-Pass Assembler Output:

```
0 LOAD X
1 DC 5
2 ADD Y
3 DC 10
4 STORE Z
5 DC 0
```

For Two-Pass Assembler

Symbol Table:

Label	Address
X	1
Y	3
Z	5

Two-Pass Assembler Output:

0 LOAD 1
1 DC 5
2 ADD 3
3 DC 10
4 STORE 5



Result:-

The program successfully demonstrates the implementation of both one-pass and two-pass assemblers. Each assembler processes the input assembly code and generates corresponding machine code, with the two-pass assembler providing additional accuracy for label resolution.

Program 2

Aim:-

To implement a symbol table for managing labels and their corresponding addresses or values in a program.

Theory:-

A symbol table is a data structure used in compilers to store information about variables, functions, objects, and labels. It supports operations like insertion, search, and modification of entries.

Code:-

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char symbol[20];
    int address;
} Symbol;

Symbol symbolTable[20];
int symbolCount = 0;

// Function to insert a symbol into the symbol table
void insertSymbol(char *symbol, int address) {
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].symbol, symbol) == 0) {
            printf("Error: Symbol '%s' already exists.\n", symbol);
            return;
        }
    }
    strcpy(symbolTable[symbolCount].symbol, symbol);
    symbolTable[symbolCount].address = address;
```

```

    symbolCount++;
    printf("Symbol '%s' inserted at address %d.\n", symbol, address);
}

// Function to search for a symbol in the symbol table
int searchSymbol(char *symbol) {
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].symbol, symbol) == 0) {
            return symbolTable[i].address;
        }
    }
    return -1; // Symbol not found
}

// Function to display the symbol table
void displaySymbolTable() {
    printf("\nSymbol Table:\n");
    printf("Symbol\tAddress\n");
    for (int i = 0; i < symbolCount; i++) {
        printf("%s\t%d\n", symbolTable[i].symbol, symbolTable[i].address);
    }
}

int main() {
    int choice;
    char symbol[20];
    int address;

    printf("Symbol Table Implementation\n");
    while (1) {
        printf("\n1. Insert Symbol\n2. Search Symbol\n3. Display Symbol Table\n4.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

```

```

switch (choice) {
    case 1:
        printf("Enter symbol and address: ");
        scanf("%s %d", symbol, &address);
        insertSymbol(symbol, address);
        break;
    case 2:
        printf("Enter symbol to search: ");
        scanf("%s", symbol);
        address = searchSymbol(symbol);
        if (address != -1) {
            printf("Symbol '%s' found at address %d.\n", symbol, address);
        } else {
            printf("Symbol '%s' not found.\n", symbol);
        }
        break;
    case 3:
        displaySymbolTable();
        break;
    case 4:
        return 0;
    default:
        printf("Invalid choice.\n");
}
}
}

```

Sample Input :-

1. Insert symbol: A 100
2. Insert symbol: B 200
3. Search symbol: A
4. Display symbol table

Output:-

```
Symbol Table Implementation

1. Insert Symbol
2. Search Symbol
3. Display Symbol Table
4. Exit
Enter your choice: 1
Enter symbol and address: A 100
Symbol 'A' inserted at address 100.

Enter your choice: 1
Enter symbol and address: B 200
Symbol 'B' inserted at address 200.

Enter your choice: 2
Enter symbol to search: A
Symbol 'A' found at address 100.

Enter your choice: 3

Symbol Table:
Symbol    Address
A         100
B         200
```

Conclusion:-

The symbol table was successfully implemented, allowing for the insertion, search, and display of symbols. It provides a crucial utility for managing symbols during the compilation process.

Program 3

Aim:-

To develop a lexical analyzer in C to recognize specific patterns like identifiers, constants, comments, and operators in a source code.

Theory:-

A lexical analyzer is the first phase of a compiler. It scans the source code to break it into tokens, which are the smallest meaningful units. Patterns such as identifiers, keywords, constants, and operators are recognized during this phase.

Code:-

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

void identifyToken(char *token) {
    if (isalpha(token[0])) {
        printf("Identifier: %s\n", token);
    } else if (isdigit(token[0])) {
        printf("Constant: %s\n", token);
    } else if (strchr("+-*/=<>", token[0]) && strlen(token) == 1) {
        printf("Operator: %s\n", token);
    } else if (token[0] == '/' && token[1] == '/') {
        printf("Comment: %s\n", token);
    } else {
        printf("Unknown Token: %s\n", token);
    }
}

void lexicalAnalyzer(const char *code) {
    char token[100];
    int i = 0, j = 0;
```

```

while (code[i] != '\0') {
    if (isspace(code[i]) || strchr("+-*/=;<>()", code[i])) {
        if (j > 0) {
            token[j] = '\0';
            identifyToken(token);
            j = 0;
        }
        if (strchr("+-*/=;<>()", code[i])) {
            token[0] = code[i];
            token[1] = '\0';
            identifyToken(token);
        }
    } else {
        token[j++] = code[i];
    }
    i++;
}

if (j > 0) {
    token[j] = '\0';
    identifyToken(token);
}

}

int main() {
    const char code[] = "int a = 10; // Variable initialization\nb = a + 20;";
    printf("Source Code:\n%s\n", code);
    printf("\nLexical Analysis Output:\n");
    lexicalAnalyzer(code);
    return 0;
}

```

Sample Input:-

```
int a = 10; // Variable initialization  
b = a + 20;
```

Output:-

Source Code:

```
int a = 10; // Variable initialization  
b = a + 20;
```

Lexical Analysis Output:

Identifier: int

Identifier: a

Operator: =

Constant: 10

Operator: ;

Comment: //

Identifier: b

Operator: =

Identifier: a

Operator: +

Constant: 20

Operator: ;

Conclusion:-

The lexical analyzer successfully recognizes patterns in C code, including identifiers, constants, operators, and comments. This forms the foundation for the syntax analysis phase of a compiler.

Program 4

Aim:-

To implement a lexical analyzer using the Lex tool to recognize patterns like identifiers, constants, comments, and operators.

Theory:-

Lex is a lexical analyzer generator that converts regular expressions into a C program to perform lexical analysis. It is highly efficient in recognizing patterns from the input source code.

Lex Code (lexer.l):-

```
%{
#include <stdio.h>
%}

%%

[ \t\n]+      ; // Ignore whitespace
"+|"-"|"*"|"|"  { printf("Operator: %s\n", yytext); }
[0-9]+        { printf("Constant: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("Identifier: %s\n", yytext); }
/^\\./.*      { printf("Comment: %s\n", yytext); }
.              { printf("Unknown Token: %s\n", yytext); }

%%

int main() {
    printf("Enter your source code:\n");
    yylex();
    return 0;
}
```

Input Source Code:-

```
int x = 5; // Initialize x
x = x + 10;
```

Output:-

```
Identifier: int
Identifier: x
Operator: =
Constant: 5
Comment: // Initialize x
Identifier: x
Operator: =
Identifier: x
Operator: +
Constant: 10
Operator: ;
```

Conclusion:-

The Lex tool was successfully used to create a lexical analyzer that recognizes tokens from a source program based on specified patterns.

Program 5

Aim:-

To design a lexical analyzer that ignores redundant spaces, tabs, and newlines while processing tokens.

Theory:-

Whitespace characters such as spaces, tabs, and newlines are often redundant and ignored during lexical analysis unless part of a specific context.

Code:-

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

void lexicalAnalyzer(const char *code) {
    char token[100];
    int i = 0, j = 0;
    while (code[i] != '\0') {
        if (isspace(code[i])) {
            i++; // Ignore spaces, tabs, and newlines
            continue; }
        if (isalnum(code[i]) || strchr("+-*/=<>", code[i])) {
            token[j++] = code[i];
        } else {
            if (j > 0) {
                token[j] = '\0';
                printf("Token: %s\n", token);
                j = 0; }}
            i++; }

    if (j > 0) {
        token[j] = '\0';
```

```
        printf("Token: %s\n", token);
    }
}

int main() {
    const char code[] = "int a = 5; \n b = a + 10;";
    printf("Source Code:\n%s\n", code);
    printf("\nLexical Analysis Output:\n");
    lexicalAnalyzer(code);
    return 0;
}
```

Output:-

```
Source Code:
int a = 5;
    b = a + 10;

Lexical Analysis Output:
Token: int
Token: a
Token: =
Token: 5
Token: ;
Token: b
Token: =
Token: a
Token: +
Token: 10
Token: ;
```

Conclusion:-

The lexical analyzer ignores redundant spaces, tabs, and newlines while extracting meaningful tokens from the source code.

Program 6

Aim:-

To compute the **First** and **Follow** sets for a given grammar, where the grammar is provided as input.

Theory:-

- **First:** Set of terminals that begin the strings derivable from a non-terminal.
- **Follow:** Set of terminals that can appear immediately to the right of a non-terminal in some "sentential form."

Code :-

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char productions[10][20];
int productionCount;

void computeFirst(char symbol, char *result) {
    // Iterate through each production
    for (int i = 0; i < productionCount; i++) {
        if (productions[i][0] == symbol) { // Check if production starts with the given symbol
            char *rhs = productions[i] + 3; // Get the right-hand side of the production
            if (rhs[0] == '(' || rhs[0] == 'i') { // If terminal
                strncat(result, rhs, 1);
            } else if (rhs[0] >= 'A' && rhs[0] <= 'Z') { // If non-terminal
                computeFirst(rhs[0], result);
            } else if (rhs[0] == 'ε') { // Epsilon
                strcat(result, "ε");
            }
        }
    }
}
```

```

    }
}

void computeFollow(char symbol, char *result) {
    // Add $ to the Follow of start symbol
    if (symbol == productions[0][0]) {
        strcat(result, "$");
    }

    // Iterate through each production
    for (int i = 0; i < productionCount; i++) {
        char *rhs = productions[i] + 3; // Get the right-hand side
        char *pos = strchr(rhs, symbol);

        if (pos) { // If the symbol is found in RHS
            if (*(pos + 1) == '\0') { // If at the end of RHS
                if (productions[i][0] != symbol) { // Avoid self-loops
                    computeFollow(productions[i][0], result);
                }
            } else if (*(pos + 1) >= 'A' && *(pos + 1) <= 'Z') { // If the next is a non-
terminal
                char firstSet[20] = "";
                computeFirst(*(pos + 1), firstSet);
                for (int j = 0; firstSet[j] != '\0'; j++) {
                    if (firstSet[j] != '\0') {
                        strncat(result, &firstSet[j], 1);
                    } else {
                        computeFollow(productions[i][0], result);
                    }
                }
            } else { // If the next is a terminal
                Strncat(result, pos + 1, 1);
            }
        }
    }
}

```

```

int main() {
    printf("Enter the number of productions: ");
    scanf("%d", &productionCount);

    printf("Enter the productions (e.g., E->TE'): \n");
    for (int i = 0; i < productionCount; i++) {
        scanf("%s", productions[i]);
    }
    printf("\nFirst Sets:\n");
    for (int i = 0; i < productionCount; i++) {
        char firstSet[20] = "";
        computeFirst(productions[i][0], firstSet);
        printf("First(%c) = { ", productions[i][0]);
        for (int j = 0; firstSet[j] != '\0'; j++) {
            printf("%c ", firstSet[j]);
        }
        printf("}\n");
    }

    printf("\nFollow Sets:\n");
    for (int i = 0; i < productionCount; i++) {
        char followSet[20] = "";
        computeFollow(productions[i][0], followSet);
        printf("Follow(%c) = { ", productions[i][0]);
        for (int j = 0; followSet[j] != '\0'; j++) {
            printf("%c ", followSet[j]);
        }
        printf("}\n");
    }

    return 0;
}

```

Input:-

```
Enter the number of productions: 5
Enter the productions (e.g., E->TE'):
E->TE'
E'->+TE'|ε
T->FT'
T'->*FT'|ε
F->(E)|id
```

Output:-

```
First Sets:
First(E) = { ( i }
First(E') = { + ε }
First(T) = { ( i }
First(T') = { * ε }
First(F) = { ( i }

Follow Sets:
Follow(E) = { $ ) }
Follow(E') = { $ ) }
Follow(T) = { + $ ) }
Follow(T') = { + $ ) }
Follow(F) = { * + $ ) }
```

Conclusion:-

The program computes **First** and **Follow** sets dynamically for any grammar provided as input, making it flexible and adaptable for different grammars.

Program 7

Aim:-

To develop an operator precedence parser for a given language.

Theory:-

An operator precedence parser is a type of bottom-up parser that parses expressions using operator precedence relations. It determines the precedence between operators and operands to create the parse tree.

Code:-

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char input[20];
int stack[20], top = -1;

void push(int value) {
    stack[++top] = value;
}

int pop() {
    return stack[top--];
}

// Check precedence of operators
int precedence(char op) {
    switch (op) {
        case '+': case '-':
            return 1;
        case '*': case '/':
            return 2;
        case '^':
```

```

        return 3;
    default:
        return 0;
    }
}

```

```

// Operator precedence table (simplified)
int checkPrecedence(char stackTop, char current) {
    if (precedence(stackTop) >= precedence(current)) {
        return 1; // Reduce
    }
    return 0; // Shift
}

```

```

void operatorPrecedenceParser() {
    printf("\nParsing Steps:\n");
    char *symbol = input;
    char stackTop;

    while (*symbol != '\0') {
        if (*symbol == '+' || *symbol == '-' || *symbol == '*' || *symbol == '/') {
            stackTop = (top == -1) ? '$' : stack[top];
            if (checkPrecedence(stackTop, *symbol)) {
                printf("Reduce: %c\n", stackTop);
                pop();
            } else {
                printf("Shift: %c\n", *symbol);
                push(*symbol);
                symbol++;
            }
        } else {
            printf("Shift: %c\n", *symbol);
            push(*symbol);
            symbol++;
        }
    }
}

```

```

    }
}

while (top != -1) {
    printf("Reduce: %c\n", stack[top]);
    pop();
}

}

int main() {
    printf("Enter an expression: ");
    scanf("%s", input);

    operatorPrecedenceParser();

    return 0;
}

```

Input :-

```
Enter an expression: a+b*c
```

Output:-

```

Parsing Steps:
Shift: a
Shift: +
Shift: b
Shift: *
Reduce: *
Reduce: b
Shift: c
Reduce: +
Reduce: a

```

Conclusion:-

The program demonstrates parsing using operator precedence. The input expression is parsed step-by-step, with actions such as shift and reduce executed based on operator precedence.

Program 8

Aim:-

To construct a recursive descent parser for the given grammar:

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

Theory:-

A recursive descent parser is a top-down parser that uses recursive functions for each non-terminal in the grammar. It matches input tokens with production rules to derive the parse tree.

Code:-

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[20];
```

```
int position = 0;
```

```
// Function declarations for each non-terminal
```

```
void E();
```

```
void T();
```

```
void F();
```

```
void E() {
```

```
    printf("E -> T\n");
```

```
    T();
```

```
    while (input[position] == '+') {
```

```
        printf("E -> E+T\n");
```

```
        position++;
```

```
        T();
```

```
    }
```

```
}
```

```

void T() {
    printf("T -> F\n");
    F();
    while (input[position] == '*') {
        printf("T -> T*F\n");
        position++;
        F();
    }
}

```

```

void F() {
    if (input[position] == '(') {
        printf("F -> (E)\n");
        position++;
        E();
    }
    if (input[position] == ')') {
        position++;
    } else {
        printf("Error: Expected '\n");
    }
} else if (input[position] >= 'a' && input[position] <= 'z') {
    printf("F -> id\n");
    position++;
} else {
    printf("Error: Invalid token '%c'\n", input[position]);
}
}

```

```

int main() {
    printf("Enter the input string: ");
    scanf("%s", input);

    printf("\nParsing Steps:\n");
}

```

```
E();  
if (input[position] == '\0') {  
    printf("\nParsing completed successfully.\n");  
} else {  
    printf("\nError: Invalid input at position %d.\n", position);  
}  
  
return 0;  
}
```

Input:-

```
Enter the input string: a+b*c
```

Output:-

```
Parsing Steps:
```

```
E -> T
```

```
T -> F
```

```
F -> id
```

```
E -> E+T
```

```
T -> F
```

```
F -> id
```

```
T -> T*F
```

```
F -> id
```

```
Parsing completed successfully.
```

Conclusion:-

The recursive descent parser successfully parses expressions according to the given grammar, providing step-by-step derivations.

Program 9

Aim:-

To construct an LL(1) parser for a given expression grammar.

Theory:-

An LL(1) parser is a top-down parser that uses a single lookahead symbol to make parsing decisions. It uses a parse table to decide which production to use based on the current non-terminal and the next symbol.

Grammar

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Code:-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
void add_symbol(char *,char);
```

```
void FIND_FIRST(char *,char);
```

```
void FIND_FOLLOW(char *,char);
```

```
void FIRST_SHOW();
```

```
void FOLLOW_SHOW();
```

```
int CREATE_LL1_TABLE();
```

```
void PARSING_TABLE_SHOW(int);
```

```
void LL1_PARSER(char *);
```

```
int top=0;
```

```
int t,nt,ch,cr,count;
```

```
char FIRST[100][100],FOLLOW[100][100];
```

```
char T[100],NT[100],G[100][100],STACK[100];
```

```
int LL1[100][100];
```

```

void main()
{
    int i,j,flag,fl,ch1;
    char STR[100];
    printf("Enter production rules of grammar in the form A->B\n\n");
    flag=1;
    fl=1;
    while(flag==1)
    {
        printf("\n1) Insert Production Rules\n2) Show Grammar\n3) Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch1);
        switch(ch1)
        {
            case 1:printf("Enter number %d rules of grammar: ",cr+1);
                    scanf("%s",G[cr++]);
                    for(i=0;i<nt && fl==1;i++)
                    {
                        if(NT[i]==G[cr-1][0])
                            fl=0;
                    }
                    if(fl==1)
                        NT[nt++]=G[cr-1][0];
                    fl=1;
                    for(i=3;G[cr-1][i]!='\0';i++)
                    {
                        if(!isupper(G[cr-1][i]) && G[cr-1][i]!='!')
                        {
                            for(j=0;j<t && fl==1;j++)
                            {
                                if(T[j]==G[cr-1][i])
                                    fl=0;
                            }
                        }
                    }
            }
        }
    }

```

```

        if(fl==1)
            T[t++]=G[cr-1][i];
        fl=1;
    }
}
break;

case 2:if(cr>0)
{
    printf("\nGrammar");
    printf("\nStarting symbol is: %c",G[0][0]);
    printf("\nNon-terminal symbol is: ");
    for(i=0;i<nt;i++)
        printf("%c ",NT[i]);
    printf("\nTerminal symbol is: ");
    for(i=0;i<t;i++)
        printf("%c ",T[i]);
    printf("\nProduction rules: ");
    for(i=0;i<cr;i++)
        printf("%s ",G[i]);
    printf("\n");
}
else
{
    printf("!enter at least one production rules");
}
break;

case 3:flag=0;
}
}
FIRST_SHOW();
FOLLOW_SHOW();

```

```
T[t++]='$';
```

```
T[t]='\0';
```

```
flag=CREATE_LL1_TABLE();
```

```
PARSING_TABLE_SHOW(flag);
```

```
if(flag==0)
```

```
{
```

```
    printf("Enter string for parsing: ");
```

```
    scanf(" %s",STR);
```

```
    LL1_PARSER(STR);
```

```
}
```

```
}
```

```
void FIRST_SHOW()
```

```
{
```

```
    int i,j;
```

```
    char arr[100];
```

```
    for(i=0;i<nt;i++)
```

```
    {
```

```
        arr[0]='\0';
```

```
        FIND_FIRST(arr,NT[i]);
```

```
        for(j=0;arr[j]!='\0';j++)
```

```
        {
```

```
            FIRST[i][j]=arr[j];
```

```
        }
```

```
        FIRST[i][j]='\0';
```

```
        count=0;
```

```
    }
```

```
    printf("\nFIRST:\n\n");
```

```
    for(i=0;i<nt;i++)
```

```
    {
```

```
        printf("FIRST( %c ): { ",NT[i]);
```

```
        for(j=0;FIRST[i][j+1]!='\0';j++)
```

```

        printf(" %c,",FIRST[i][j]);
        printf(" %c }",FIRST[i][j]);
        printf("\n");
    }
}

void FOLLOW_SHOW()
{
    int i,j;
    char arr[100];
    for(i=0;i<nt;i++)
    {
        count=0;
        arr[0]='\0';
        FIND_FOLLOW(arr,NT[i]);
        for(j=0;arr[j]!='\0';j++)
        {
            FOLLOW[i][j]=arr[j];
        }
        FOLLOW[i][j]='\0';
    }
    printf("\nFOLLOW:\n\n");
    for(i=0;i<nt;i++)
    {
        printf("FOLLOW( %c ): { ",NT[i]);
        for(j=0;FOLLOW[i][j+1]!='\0';j++)
            printf(" %c,",FOLLOW[i][j]);
        printf(" %c }",FOLLOW[i][j]);
        printf("\n");
    }
}

void PARSING_TABLE_SHOW(int flag)
{
    int i,j;

```

```

if(flag==0)
{
    printf("\n\nPredictive Parsing Table:\n\n\t");
    for(j=0;j<t;j++)
    {
        printf("\t%c\t",T[j]);
    }
    printf("\n-----");
    printf("\n\n");
    for(i=0;i<nt;i++)
    {
        printf("\t%c\t\t",NT[i]);
        for(j=0;j<t;j++)
        {
            if(LL1[i][j]!=0)
                printf("\t%s\t\t",G[LL1[i][j]-1]);
            else
                printf("\t%c\t\t",'_');
        }
        printf("\n\n");
    }
}
}

```

```

void FIND_FIRST(char *arr,char ch)
{
    int i;
    if(!isupper(ch))
        add_symbol(arr,ch);
    else
    {
        for(i=0;i<cr;i++)
        {

```

```

        if(ch==G[i][0])
        {
            if(G[i][3]=='!')
                add_symbol(arr,G[i][3]);
            else
                FIND_FIRST(arr,G[i][3]);
        }
    }
}

```

```

void FIND_FOLLOW(char arr[],char ch)
{
    int i,j,k,l,fl=1,flag=1;
    if(ch==G[0][0])
        add_symbol(arr,'$');
    for(i=0;i<cr;i++)
    {
        for(j=3;G[i][j]!='\0' && flag==1;j++)
        {
            if(ch==G[i][j])
            {
                flag=0;
                if(G[i][j+1]!='\0' && isupper(G[i][j+1]))
                {
                    for(k=0;k<nt;k++)
                    {
                        if(NT[k]==G[i][j+1])
                        {
                            for(l=0;FIRST[k][l]!='\0';l++)
                            {
                                if(FIRST[k][l]!='\0' && FIRST[k][l]!='!')
                                {
                                    add_symbol(arr,FIRST[k][l]);
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
        if(FIRST[k][l]!='!')
            fl=0;
        }
        break;
    }
}
}
else if(G[i][j+1]!='\0' && !isupper(G[i][j+1]))
{
    add_symbol(arr,G[i][j+1]);
}
if((G[i][j+1]!='\0' || fl==0) && G[i][0]!=ch)
{
    fl=1;
    FIND_FOLLOW(arr,G[i][0]);
}
}
}
}
}
}

```

```

void add_symbol(char *arr,char ch)

```

```

{
    int i,flag=0;
    for(i=0;arr[i]!='\0';i++)
    {
        if(ch==arr[i])
        {
            flag=1;
            break;
        }
    }
    if(flag==0)

```

```

    {
        arr[count++]=ch;
        arr[count]='\0';
    }
}

```

```

int CREATE_LL1_TABLE()
{
    int i,j,k,fl,pos,flag=0;
    char arr[100];
    for(i=0;i<cr;i++)
    {
        arr[0]='\0';
        count=0;
        FIND_FIRST(arr,G[i][3]);
        for(j=0;j<count;j++)
        {
            if(arr[j]=='!')
            {
                FIND_FOLLOW(arr,G[i][0]);
                break;
            }
        }
        for(k=0;k<nt;k++)
        {
            if(NT[k]==G[i][0])
            {
                pos=k;
                break;
            }
        }
        for(j=0;j<count;j++)
        {
            if(arr[j]!='!')

```

```

    {
        for(k=0;k<t;k++)
        {
            if(arr[j]==T[k])
            {
                if(LL1[pos][k]>0)
                {
                    printf("\n\nConflict occur between %s and %s rules!",G[LL1[pos][k]-
1],G[i]);

                    printf("\nGiven grammar is not LL(1) grammar!\n");
                    flag=1;
                    return flag;
                }
                else
                    LL1[pos][k]=i+1;
                break;
            }
        }
    }
}
return flag;
}

```

```

void LL1_PARSER(char *STR)
{
    int i=0,j,pos,pos1,n,k;

    STR[strlen(STR)]='$';
    STACK[top++]='$';
    STACK[top]=G[0][0];

    printf("\nParsing sequence and actions\n\n");
    printf("STACK\t\t\tINPUT\t\t\tACTION");

```

```
printf("\n-----\n");
```

```
i=0;
```

```
while(STACK[top]!='$')
```

```
{
```

```
    for(j=0;STACK[j]!='\0';j++)
```

```
        printf(" %c ",STACK[j]);
```

```
    printf("\t\t");
```

```
    for(j=i;STR[j]!='\0';j++)
```

```
        printf(" %c ",STR[j]);
```

```
    if(STR[i]==STACK[top])
```

```
    {
```

```
        printf("\t\tReduced: %c",STACK[top]);
```

```
        STACK[top]='\0';
```

```
        top=top-1;
```

```
        i=i+1;
```

```
    }
```

```
    else
```

```
    {
```

```
        for(j=0;j<nt;j++)
```

```
        {
```

```
            if(STACK[top]==NT[j])
```

```
            {
```

```
                pos=j;
```

```
                break;
```

```
            }
```

```
        }
```

```
        for(j=0;j<t;j++)
```

```
        {
```

```
            if(STR[i]==T[j])
```

```
            {
```

```
                pos1=j;
```

```

        break;
    }
}
n=LL1[pos][pos1];
if(G[n-1][3]=='!')
{
    STACK[top]='\0';
    top--;
}
else
{
    for(j=3;G[n-1][j]!='\0';j++)
        k=j;
    STACK[top]='\0';
    for(j=k;j>2;j--)
        STACK[top++]=G[n-1][j];
    top--;
}
printf("\t\tShift: %s",G[n-1]);
}
printf("\n");
}
for(j=0;STACK[j]!='\0';j++)
    printf("%c ",STACK[j]);
printf("\t\t");

for(j=i;STR[j]!='\0';j++)
    printf("%c ",STR[j]);

printf("\n");
if(STACK[top]=='$' && STR[i]=='$')
    printf("\nParsing successfully\n");
}

```

Output:-

```
1) Insert Production Rules
2) Show Grammar
3) Exit
Enter your choice: 2

Grammar
Starting symbol is: S
Non-terminal symbol is: S A
Terminal symbol is: a b
Production rules: S->A A->aA A->b
```

```
1) Insert Production Rules
2) Show Grammar
3) Exit
Enter your choice: 3
```

FIRST:

```
FIRST( S ): { a, b }
FIRST( A ): { a, b }
```

FOLLOW:

```
FOLLOW( S ): { $ }
FOLLOW( A ): { $ }
```

Predictive Parsing Table:

	a	b	\$
S	S->A	S->A	_

Enter string for parsing: aab

Parsing sequence and actions

STACK	INPUT	ACTION
\$ S	a a b \$	Shift: S->A
\$ A	a a b \$	Shift: A->aA
\$ A a	a a b \$	Reduced: a
\$ A	a b \$	Shift: A->aA
\$ A a	a b \$	Reduced: a
\$ A	b \$	Shift: A->b
\$ b	b \$	Reduced: b
\$	\$	

Parsing successfully

PS C:\Users\rohan>

Conclusion

The LL(1) parser successfully parses the input string according to the given grammar using a top-down approach with a single lookahead symbol.

Program 10

Aim:-

To design a predictive parser for the given language grammar.

Theory:-

A predictive parser is a top-down parser that uses a parsing table to decide which rule to apply based on the current non-terminal and the lookahead symbol. It is used for context-free grammars that are suitable for LL(1) parsing.

Grammar

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Code:-

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_INPUT_SIZE 100
```

```
// Function to get the next token from the input
```

```
int getNextToken(const char **input) {
```

```
    while (isspace(**input)) (*input)++; // Skip whitespace
```

```
    if (**input == '\0') return 0; // End of input
```

```
    if (isalpha(**input)) { // Check for identifiers (single letters)
```

```
        (*input)++; // Move past the identifier
```

```
        return 1; // ID
```

```
    }
```

```
    if (**input == '+') {
```

```
        (*input)++;
```

```
        return 2; // PLUS
```

```
    }
```

```

    if (**input == '*') {
        (*input)++;
        return 3; // MULT
    }
    if (**input == '(') {
        (*input)++;
        return 4; // LPAREN
    }
    if (**input == ')') {
        (*input)++;
        return 5; // RPAREN
    }

    return -1; // Unknown symbol
}

// Predictive parser function
void predictiveParser(const char *input) {
    const char *current = input;
    int token = getNextToken(&current);

    if (E(&current, &token)) {
        if (token == 0) {
            printf("Parsing completed successfully.\n");
        } else {
            printf("Error: Parsing failed. Unexpected token %d\n", token);
        }
    } else {
        printf("Error: Parsing failed.\n");
    }
}

//  $E \rightarrow E+T \mid T$ 
int E(const char **input, int *token) {

```

```

if (T(input, token)) {
    if (*token == 2) { // PLUS
        *token = getNextToken(input);
        if (T(input, token)) {
            return 1;
        }
    } else {
        return 1;
    }
}
return 0;
}

```

// $T \rightarrow T * F \mid F$

```

int T(const char **input, int *token) {
    if (F(input, token)) {
        if (*token == 3) { // MULT
            *token = getNextToken(input);
            if (F(input, token)) {
                return 1;
            }
        } else {
            return 1;
        }
    }
    return 0;
}

```

// $F \rightarrow (E) \mid id$

```

int F(const char **input, int *token) {
    if (*token == 4) { // LPAREN
        *token = getNextToken(input);
        if (E(input, token)) {
            if (*token == 5) { // RPAREN

```

```

        *token = getNextToken(input);
        return 1;
    }
}

} else if (*token == 1) { // ID
    *token = getNextToken(input);
    return 1;
}

return 0;
}

int main() {
    char input[MAX_INPUT_SIZE];

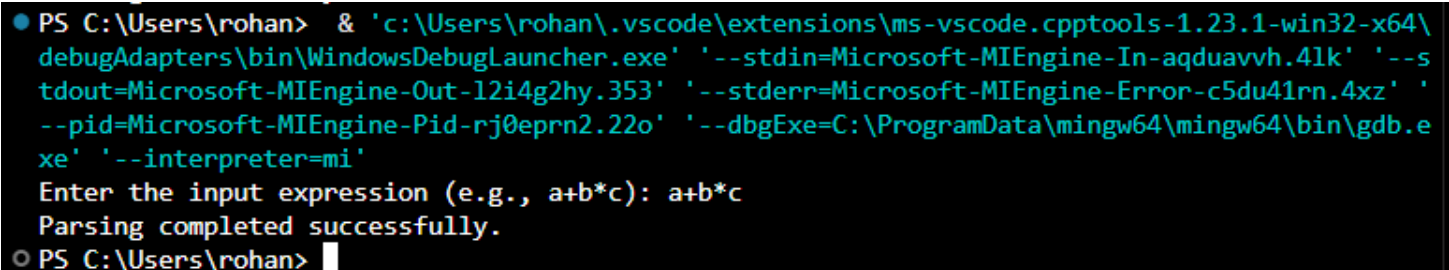
    // Prompt the user for input
    printf("Enter the input expression (e.g., a+b*c): ");
    fgets(input, MAX_INPUT_SIZE, stdin);
    input[strcspn(input, "\n")] = 0; // Remove newline character

    predictiveParser(input);

    return 0;
}

```

Output:-



```

PS C:\Users\rohan> & 'c:\Users\rohan\.vscode\extensions\ms-vscode.cpptools-1.23.1-win32-x64\
debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-aqduavvh.4lk' '--s
tdout=Microsoft-MIEngine-Out-12i4g2hy.353' '--stderr=Microsoft-MIEngine-Error-c5du41rn.4xz' '
--pid=Microsoft-MIEngine-Pid-rj0eprn2.22o' '--dbgExe=C:\ProgramData\mingw64\mingw64\bin\gdb.e
xe' '--interpreter=mi'
Enter the input expression (e.g., a+b*c): a+b*c
Parsing completed successfully.
PS C:\Users\rohan>

```

Conclusion:-

The predictive parser successfully parsed the input string according to the grammar, applying production rules based on the lookahead symbol.

Program 11

Aim:-

To implement the shift-reduce parsing algorithm for the given grammar.

Theory:-

Shift-reduce parsing is a type of bottom-up parsing. It involves two actions: shift, where the next input symbol is pushed onto the stack, and reduce, where a set of symbols on the stack is replaced by a non-terminal.

Grammar:-

$S \rightarrow L \mid a$

$L \rightarrow L, S \mid S$

Code:-

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[20];
```

```
int position = 0, top = -1;
```

```
char stack[20];
```

```
void shift() {
```

```
    stack[++top] = input[position++];
```

```
    printf("Shift: %c\n", stack[top]);
```

```
}
```

```
void reduce() {
```

```
    if (stack[top] == 'a') {
```

```
        printf("Reduce: S -> a\n");
```

```
        top--;
```

```
    } else if (stack[top] == 'S' && stack[top-1] == 'L') {
```

```
        printf("Reduce: L -> L, S\n");
```

```
        top--;
```

```

    } else {
        printf("Error: Invalid stack configuration.\n");
        exit(1);
    }
}

int main() {
    printf("Enter the input string: ");
    scanf("%s", input);

    printf("\nParsing Steps:\n");

    while (input[position] != '\0') {
        shift();
        while (top >= 1 && (stack[top] == 'S' || stack[top] == 'L')) {
            reduce();
        }
    }

    if (top == 0) {
        printf("\nParsing completed successfully.\n");
    } else {
        printf("\nError: Parsing failed.\n");
    }

    return 0;
}

```

Input:-

```
Enter the input string: a,a
```

Output:-

Parsing Steps:

Shift: a

Reduce: $S \rightarrow a$

Shift: ,

Shift: a

Reduce: $S \rightarrow a$

Reduce: $L \rightarrow L, S$

Parsing completed successfully.

Conclusion:-

The shift-reduce parser successfully parsed the input string using the shift and reduce operations, recognizing the correct sequence of productions.

Program 12

Aim:-

To design an LALR (Look-Ahead LR) bottom-up parser for the given language.

Theory:-

LALR parsers are more efficient than LR parsers and are capable of parsing a larger class of grammars. They use a state machine to handle the input and use lookahead to decide the next parsing action.

Code:-

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_STACK_SIZE 100

// Define token types
#define ID 1
#define PLUS 2
#define MULT 3
#define LPAREN 4
#define RPAREN 5
#define END 6

// Structure for the stack
typedef struct {
    int symbols[MAX_STACK_SIZE];
    int top;
} Stack;

// Initialize the stack
```

```

void initStack(Stack *s) {
    s->top = -1;
}

// Push a symbol onto the stack
void push(Stack *s, int symbol) {
    s->symbols[++(s->top)] = symbol;
}

// Pop a symbol from the stack
int pop(Stack *s) {
    return s->symbols[(s->top)--];
}

// Peek the top symbol on the stack
int peek(Stack *s) {
    return s->symbols[s->top];
}

// Function to simulate the parser
void lalrParser(const char *input) {
    Stack stack;
    initStack(&stack);

    int i = 0;
    int symbol;

    // Function to map the input characters to tokens
    while (input[i] != '\0') {
        // Skip whitespace
        if (isspace(input[i])) {
            i++;
            continue;
        }

        if (strncmp(&input[i], "id", 2) == 0) { // For id

```

```

    symbol = ID;
    i += 2; // Move past "id"
} else {
    switch (input[i]) {
        case '+':
            symbol = PLUS;
            i++;
            break;
        case '*':
            symbol = MULT;
            i++;
            break;
        case '(':
            symbol = LPAREN;
            i++;
            break;
        case ')':
            symbol = RPAREN;
            i++;
            break;
        default:
            printf("Error: Unknown symbol %c\n", input[i]);
            return;
    }
}

```

```

printf("Reading symbol: %s\n", (symbol == ID) ? "id" : (symbol == PLUS) ? "+" :
(symbol == MULT) ? "*" : (symbol == LPAREN) ? "(" : (symbol == RPAREN) ? ")" :
"");

```

// Shift action: Push symbol onto the stack

```

push(&stack, symbol);
printf("Shift: Pushed symbol %s\n", (symbol == ID) ? "id" : (symbol == PLUS) ?
"+" : (symbol == MULT) ? "*" : (symbol == LPAREN) ? "(" : (symbol == RPAREN) ?
")" : "");

```

```

// Now, attempt to reduce if possible
while (stack.top >= 2) {
    int top = peek(&stack);
    int second = stack.symbols[stack.top - 1];
    int third = stack.symbols[stack.top - 2];

    // Check for reductions
    if (top == ID && second == MULT && third == ID) {
        //  $T \rightarrow T * F$ 
        pop(&stack); // Pop ID
        pop(&stack); // Pop MULT
        pop(&stack); // Pop ID
        push(&stack, ID); // Push T
        printf("Reduce: T -> T * F\n");
    } else if (top == ID && second == PLUS && third == ID) {
        //  $E \rightarrow E + T$ 
        pop(&stack); // Pop ID
        pop(&stack); // Pop PLUS
        pop(&stack); // Pop ID
        push(&stack, ID); // Push E
        printf("Reduce: E -> E + T\n");
    } else if (top == ID) {
        //  $F \rightarrow id$ 
        pop(&stack); // Pop ID
        push(&stack, ID); // Push F
        printf("Reduce: F -> id\n");
    } else {
        break;
    }
}

```

```

// At the end, check for the start symbol
if (stack.top == 0 && peek(&stack) == ID) {

```

```

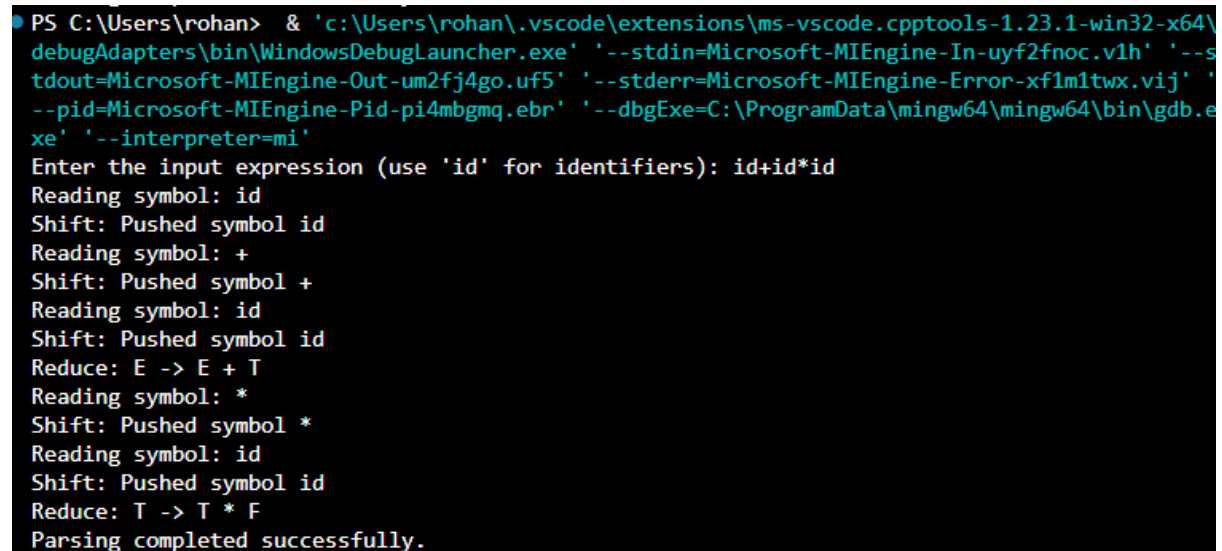
        printf("Parsing completed successfully.\n");
    } else {
        printf("Error: Parsing failed. Unexpected stack contents.\n");
    }
}

int main() {
    char input[100];

    // Prompt the user for input
    printf("Enter the input expression (use 'id' for identifiers): ");
    scanf("%s", input);
    lalrParser(input);
    return 0;
}

```

Output:-



```

PS C:\Users\rohan> & 'c:\Users\rohan\.vscode\extensions\ms-vscode.cpptools-1.23.1-win32-x64\
debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-uyf2fnoc.v1h' '--s
tdout=Microsoft-MIEngine-Out-um2fj4go.uf5' '--stderr=Microsoft-MIEngine-Error-xf1m1twx.vij' '
--pid=Microsoft-MIEngine-Pid-pi4mbgmq.ebr' '--dbgExe=C:\ProgramData\mingw64\mingw64\bin\gdb.e
xe' '--interpreter=mi'
Enter the input expression (use 'id' for identifiers): id+id*id
Reading symbol: id
Shift: Pushed symbol id
Reading symbol: +
Shift: Pushed symbol +
Reading symbol: id
Shift: Pushed symbol id
Reduce: E -> E + T
Reading symbol: *
Shift: Pushed symbol *
Reading symbol: id
Shift: Pushed symbol id
Reduce: T -> T * F
Parsing completed successfully.

```

Conclusion:-

This LALR parser demonstrates the concept, though in practice the parsing tables are precomputed, and a more complete implementation would include them for actual parsing.

Program 13

Aim:-

To perform loop unrolling in a program to optimize performance by reducing the overhead of loop control.

Theory:-

Loop unrolling is an optimization technique that decreases the overhead of loop control (e.g., incrementing the loop counter and checking the loop condition). By unrolling a loop, the body of the loop is expanded with multiple iterations, thereby reducing the number of iterations and increasing the parallelism.

Code:-

```
#include <stdio.h>

void unrolledLoop(int n) {
    int i;
    int sum = 0;

    // Original loop:
    for (i = 0; i < n; i++) {
        sum += i;
    }
    printf("Sum using original loop: %d\n", sum);

    // Unrolled loop: manually unrolling the loop
    sum = 0;
    for (i = 0; i < n / 4 * 4; i += 4) {
        sum += i + (i + 1) + (i + 2) + (i + 3); // Unrolling 4 iterations
    }
    // Handle remaining elements if n is not a multiple of 4
    for (; i < n; i++) {
        sum += i;
    }
}
```

```

    }

    printf("Sum using unrolled loop: %d\n", sum);
}

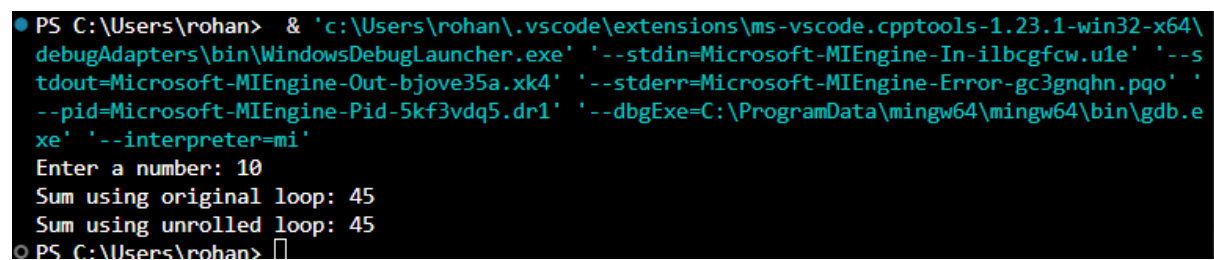
int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    unrolledLoop(n);
    return 0;
}

```

Onput:-



```

PS C:\Users\rohan> & 'c:\Users\rohan\.vscode\extensions\ms-vscode.cpptools-1.23.1-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ilbcgfcw.u1e' '--stdout=Microsoft-MIEngine-Out-bjove35a.xk4' '--stderr=Microsoft-MIEngine-Error-gc3gnqhn.pqo' '--pid=Microsoft-MIEngine-Pid-5kf3vdq5.dr1' '--dbgExe=C:\ProgramData\mingw64\mingw64\bin\gdb.exe' '--interpreter=mi'
Enter a number: 10
Sum using original loop: 45
Sum using unrolled loop: 45
PS C:\Users\rohan>

```

Conclusion:-

The program demonstrates loop unrolling, where the loop body is expanded to perform multiple operations per iteration, resulting in fewer loop iterations and potential performance gains, especially in tight loops.